

SCI CPU Optimization for ROCK 5 Model A

Technical deployment report covering the original SCI inference path, CPU optimizations, measured host performance, output fidelity, and ROCK 5A validation procedure.

Technical summary

The packaged TorchScript graph is the recommended CPU backend. On the development host it reduced median 600x400 model latency from **2.510 ms to 1.072 ms** at four threads, a **2.34x speedup**, while increasing implied model-only throughput from **398.4 to 933.3 FPS**. The largest floating-point difference from the original repository model was **4.17e-7**, so the optimization is functionally equivalent. These timings are x86 evidence, not a ROCK 5A forecast; the included benchmark must be run on the board for deployable ARM numbers.

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Optimized latency

1.07

Median model-only latency at 600x400 with four CPU threads.

Original latency 2.51

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Speedup

2.34

Relative reduction in model execution time versus the eager baseline.

Latency reduction +57.3%

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Optimized throughput

933.27

Throughput implied by median model-only latency; file I/O is excluded.

Original throughput 398.36

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Maximum output error

0.417 ppm

Maximum absolute floating-point difference from the original repository model.

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Deployment parameters

252

Trainable scalar parameters remaining after BatchNorm fusion.

Source: TorchScript export validation

Table: model_info.json

TorchScript delivered the largest measured CPU gain

TorchScript was faster than both eager PyTorch and ONNX Runtime at every tested thread count on the development host. The gain remained substantial from one to four threads, showing that graph simplification and freezing improved

execution independently of thread-level parallelism.

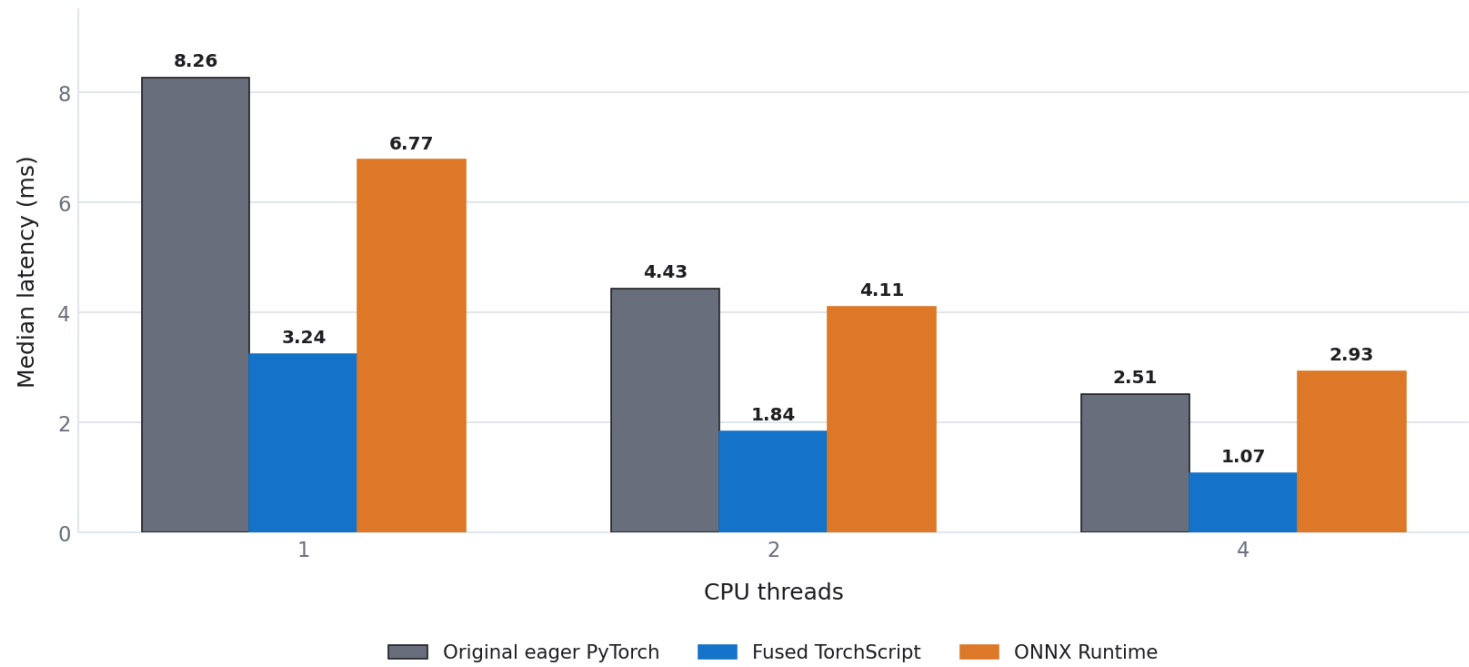
Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Median inference latency by CPU threads and backend

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv



How to read the latency chart

Each cluster holds the same image, warm-up count, measured-run count, and CPU thread count. Lower bars are faster. Four-thread TorchScript reached the lowest median at **1.072 ms**. ONNX Runtime improved on eager execution at one and two threads but was slower than eager at four threads on this host, which is why TorchScript remains the default while both backends are shipped for ARM benchmarking.

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Complete CPU benchmark results

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Threads	Backend	Median (ms)	Mean (ms)	P95 (ms)	FPS	Speedup vs eager
4	Fused TorchScript	1.07	1.14	1.59	933.27	+2.34
2	Fused TorchScript	1.84	1.87	2.11	544.72	+2.41
4	Original eager PyTorch	2.51	2.74	3.7	398.36	+1
4	ONNX Runtime	2.93	3.02	3.94	340.87	+0.86
1	Fused TorchScript	3.24	3.29	3.58	308.9	+2.55
2	ONNX Runtime	4.11	4.16	4.67	243.46	+1.08
2	Original eager PyTorch	4.43	4.51	5.01	225.91	+1
1	ONNX Runtime	6.77	6.79	7.36	147.65	+1.22
1	Original eager PyTorch	8.26	8.29	8.69	121.03	+1

Deployment artifacts are 88% to 94% smaller than the checkpoint

The original 42.94 KiB checkpoint contains training-only calibration state and duplicated module registrations. The ready-to-run artifact is **5.27 KiB in TorchScript** or **2.55 KiB in ONNX**, reducing copy size by **87.7%** and **94.1%**, respectively.

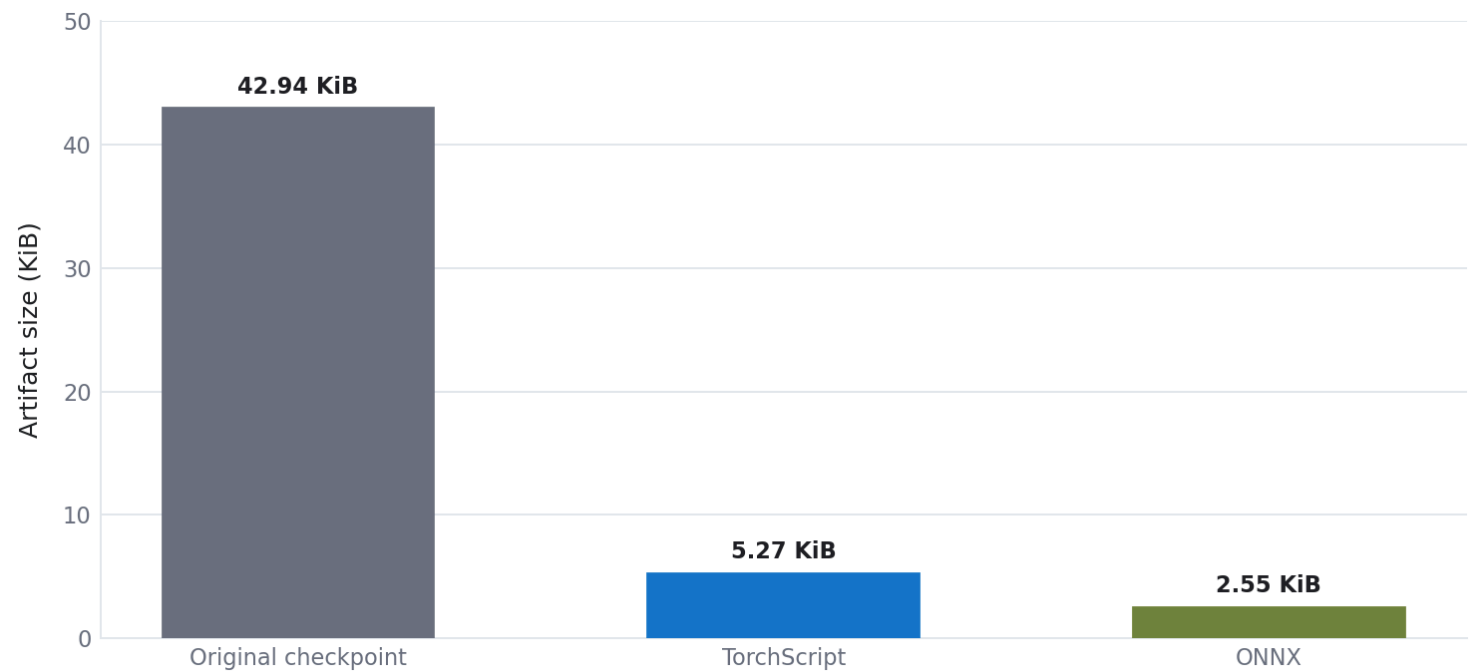
Source: Deployment artifact file inventory

Table: report_support/file_inventory.csv

Model artifact size before and after deployment optimization

Source: Deployment artifact file inventory

Table: report_support/file_inventory.csv



How to read the artifact-size chart

The bars use a zero baseline and show exact on-disk KiB. The reduction comes from deploying only the single SCI enhancement pass and serializing the inference graph. It lowers storage and transfer overhead; runtime memory is still dominated by image feature maps and the selected inference library.

Source: Deployment artifact file inventory

Table: report_support/file_inventory.csv

The deployment graph removes training work without changing the learned enhancer

SCI trains with iterative stages and a self-calibration network, but its intended inference path uses one shared illumination-estimation block. The standalone package keeps the three 3x3 convolutions used by `Finetunemodel1`, folds BatchNorm into the middle convolution, freezes the graph, uses channels-last tensors, performs safe arithmetic in place, and runs under inference mode.

Source: Standalone SCI deployment implementation

File: model_def.py

Deployment changes compared with the original repository path

Source: Before-and-after deployment change log

Table: report_support/change_log.csv

Step	Before	After	CPU effect	Quality effect
1	Training model imported loss and calibration code	Inference-only module with no training dependency	Less initialization and package complexity	None; training-only path removed
2	BatchNorm executed after the middle convolution	BatchNorm folded into convolution weights and bias	One runtime operator and tensor pass removed	Equivalent up to floating-point rounding
3	Python eager module dispatch	Scripted and frozen TorchScript graph	Lower dispatch overhead and constant folding	None beyond floating-point rounding
4	Default contiguous NCHW image conversion	Channels-last tensor strides from HWC input	Avoids an extra layout conversion and suits Arm kernels	None
5	Out-of-place temporary tensors	Safe in-place activation and clamp operations	Lower allocation and memory traffic	None beyond operation-order rounding
6	OS chooses all CPU cores and default thread pools	Four threads with automatic fastest-cluster affinity	Targets RK3588S Cortex-A76 cores and reduces migration	None
7	One PyTorch execution option	TorchScript plus optional ONNX Runtime	Allows board-specific backend selection	Numerically validated
8	High default PNG compression	Low-compression PNG output	Less end-to-end encoding work	Lossless pixels are unchanged

Scope and metric definitions

Cohort: one 600x400 RGB low-light image from the included LOL evaluation sample. **Latency:** wall-clock time for model execution only; Pillow decode, tensor creation, and PNG encoding are excluded. **Median, mean, and p95:** calculated from 100 measured runs after 20 warm-up runs. **FPS:** 1000 divided by median milliseconds and therefore an implied single-stream model-only rate, not camera-pipeline throughput. **Speedup:** eager median divided by optimized median at the same thread count.

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Measurement and validation methodology

The baseline reconstructs the repository's Finetunemodel enhancement path from Model_V3.pt. The optimized model folds BatchNorm analytically, scripts and freezes a dynamic-resolution graph, and preserves channels-last memory layout. Thread counts 1, 2, and 4 were evaluated independently. Numerical outputs were compared before 8-bit conversion; saved PNGs were then compared per pixel. The packaged benchmark.py repeats the same method and records platform, architecture, Python, PyTorch, affinity, resolution, run count, and backend results in JSON and CSV.

Source: Measured CPU benchmark results

Table: report_support/benchmark_aggregate.csv

Robustness checks show visually equivalent output

The deployment graph was checked against the original repository class, not only against a rewritten baseline. Maximum and mean floating-point errors remained far below one 8-bit intensity step. The saved comparison images are retained in results/ for visual audit.

Source: Measured CPU benchmark results

Output-equivalence checks

Source: Numerical and pixel fidelity checks

Table: report_support/fidelity_checks.csv

Check	Measured result	Interpretation
Changed PNG pixels	5 of 240,000	0.0021% of pixels
Export validation shapes	96x64, 320x240, 600x400	Dynamic spatial dimensions verified
Largest saved-pixel difference	1 level in one channel	Visually indistinguishable
Maximum floating-point error	4.17×10^{-7}	Far below one 8-bit intensity step (1/255)
Mean floating-point error	3.91×10^{-8}	No material aggregate drift

Limits and uncertainty

- The reported latency is from a Windows x86 development machine and must not be presented as ROCK 5A performance.
- ROCK 5A results will vary with the Linux image, PyTorch and ONNX Runtime builds, CPU governor, thermal state, memory pressure, and power delivery.
- The package is CPU-only as requested; the RK3588S NPU and Mali GPU were not evaluated.
- Model-only FPS excludes camera capture, image conversion, resizing, and output encoding.
- ONNX and TorchScript can rank differently on Arm, so the faster backend must be selected using board measurements.

Recommended ROCK 5A validation

1. Copy the complete SBC inference folder to a 64-bit ROCK 5A installation.
2. Run `./install_rock5a.sh`, then confirm `uname -m` reports `aarch64`.
3. Run `benchmark.py` at 1, 2, and 4 threads with automatic A76-core affinity.
4. Select TorchScript or ONNX using median latency while checking p95 stability.
5. Repeat under the real camera resolution and sustained workload while monitoring temperature and throttling.
6. Keep full resolution by default; use `--max-side` only when the application explicitly accepts a resolution-speed tradeoff.

Further questions

- Which ROCK 5A OS image and Python version will be used in production?
- What is the real input resolution, frame rate, and camera interface?
- Is the objective minimum single-frame latency, maximum batch throughput, or a fixed real-time FPS target?
- After CPU validation, would an optional RKNN/NPU build be acceptable as a separate deployment mode?